🎯 Goal

Snag power sources to grow your snake-like robots and make sure you have the biggest bots standing.

✔ Rules

The game is played on a **grid**.

Each player controls a team of **snakebots**. On each turn, the snakebots move **simultaneously** according to the players' commands.

🗺 Map

The grid is seen from the side, and is made up of **platforms**. Platforms are impassable cells.

On this grid you may find parts of a snakebot's **body** and **power sources**.

🐍 Snakebot

Snakebots are multiple adjacent cell-sized body parts. The first cell being their **head**.

Snakebots are affected by **gravity**. Meaning **at least one body** part must be above something **solid** or they will **fall**.

Other snakebots are considered **solid**, as well as **platforms** and **power sources** .

🔄 Movement

Snakebots are **perpetually moving**, and will on each turn head in the last direction they were facing unless given a new direction to turn by the player.

The starting direction is **up**.

When moving, a snakebot will advance their head in the direction it is facing, and the rest of its body parts will follow.

Case 1: If the cell the snakebot's head has moved into contains a **platform** or other **body part**, the snakebot's head is destroyed and the next part in its body becomes the **new head**. This only happens if is has at least three remaining parts. If not, the whole snakebot is removed.

Case 2: If the cell the snakebot has moved into contains a **power source**, the snakebot will **eat it**. This has two effects:

1. The snakebot **grows**, a new body part appears at the end of its body.
2. This cell is no longer considered **solid**.

These collisions are resolved simultaneously for all snakebots.

Special case: If multiple snakebot heads collide on a cell containing a **power source**, that power source is considered **eaten by each of those snakebots**!

Once movement and removals are resolved, the snakebots all **fall** downwards until a body part lands on something **solid**.

It is possible to extend your **snakebot** beyond the borders of the grid. But falling out of the playing area will result in the snakebot being removed.

🎬 Actions

Each turn, players must provide at least one action on the standard output.

Actions must be separated by a semicolon `;` and be one of the following.

Commands to move snakebot with id `id`:

- `id UP`: sets a snakebot's direction to UP `(x,y)=(0,-1)`.
- `id DOWN`: sets a snakebot's direction to DOWN `(x,y)=(0,1)`.
- `id LEFT`: sets a snakebot's direction to LEFT `(x,y)=(-1,0)`.
- `id RIGHT`: sets a snakebot's direction to RIGHT `(x,y)=(1,0)`.

Any of the movement actions can be followed by text which will be displayed above the appropriate snakebot for **debugging** purposes.

Special commands:

- `MARK x y`: places a marker at the specified coordinates. Markers are visible in the viewer for **debugging** purposes.
- `WAIT` : do nothing.

Game End

The game is over if any of these are true at the end of a turn:

- All of a player's snakebots have been removed.
- There are no more **power sources** left to eat.
- 200 turns have passed.

Victory Conditions

Have more total body parts across all your snakebots than your opponent at the end of the game.

Defeat Conditions

Your program does not provide a command in the alloted time or one of the commands is invalid.

Technical Details

- The source code of this game will be published to **this GitHub repo**.

Debugging tips

- Use the `MARK x y` action to highlight up to 4 cells per turn.
- Hover over the grid to see extra information on the cell under your mouse.
- Press the gear icon on the viewer to access extra display options.
- Use the keyboard to control the action: space to play/pause, arrows to step 1 frame at a time.

Click to expand

Game Protocol

Initialization Input

Line 1: `myId`, an integer for your player identification (`0` or `1`).
Line 2: `width`, an integer for the width of the grid.
Line 3: `height` an integer for the height of the grid.

Next `height` **lines:** One row of the grid containing `width` characters each. Each character can be:

- `#`: a **platform**.
- `.`: a **free** cell.

Next line: `snakbotsPerPlayer` an integer for amount of snakebots each player controls.

Next `snakbotsPerPlayer` **lines:** One integer `snakebotId` for each of the snakebots controlled by **you**.

Next `snakbotsPerPlayer` **lines:** One integer `snakebotId` for each of the snakebots controlled by **your opponent**.

Input for one game turn

First line: `powerSourceCount`, one integer for the number of remaining **power sources** on the grid.

Next `powerSourceCount` **lines:** The following `2` integers for each power source:

- `x`: X coordinate
- `y`: Y coordinate

Next line: `snakebotCount`, one integer for the number of remaining **snakebots** on the grid.

Next `snakebotCount` **lines:** The following `2` inputs for each snakebot:

- `snakebotId`: an integer for this snakebot's identifier
- `body`: a string of colon (`:`) separated coordinates, each coordinate being the position of a body part with the form "`x,y`". The first coordinate in the list is the **head**.
  Example: "`0,1:1,1:2,1`" *is a snakebot with a head at* `x=0` *and* `y=1` *with two more body parts to the right.*

Output

A single line, containing at least one action:

- `id UP` where id is the `snakebotId` of a snakebot you control.
- `id DOWN` where id is the `snakebotId` of a snakebot you control.
- `id LEFT` where id is the `snakebotId` of a snakebot you control.
- `id RIGHT` where id is the `snakebotId` of a snakebot you control.
- `MARK x y`: you may mark up to **four** coordinates per turn.
- `WAIT`

Instructions are separated by semi-columns (`;`). For example:

```
1 LEFT;2 RIGHT;MARK 12 2
```

Constraints

Response time per turn ≤ `50` ms
Response time for the first turn ≤ `1000` ms
`15` ≤ `width` ≤ `45`
`10` ≤ `height` ≤ `30`
`1` ≤ `snakebotCount` ≤ `8`